

SDP



Copilot Memory

Stateful AI That Remembers Your Preferences

Stop re-explaining your coding style every session

"How do I stop re-explaining my coding style every session?"

Every developer using Copilot hits the same wall:

The AI forgets everything when you close the chat.

Daily Copilot Users

Developers who open VS Code every day and reset Copilot context from scratch each time — forfeiting continuity between sessions.

5–10 min

Time spent re-establishing context per session for most developers who do not use persistent memory.

Team Leads and Architects

Leads enforcing team patterns and style guides who want Copilot aligned by default — not just when someone remembers to re-...

2 minutes

Time to write the preferences that eliminate that re-establishment cost permanently — a one-time investment.

Platform and DevOps Engineers

Engineers with strong tool and workflow preferences who want Copilot to adapt to their stack without manual prompting every...

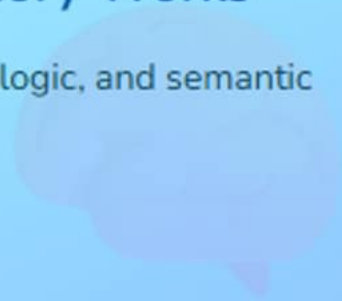
Semantic retrieval

Memory activates by meaning — not keywords. Writing once covers every future session that shares the same intent.

PART 1

How Copilot Memory Works

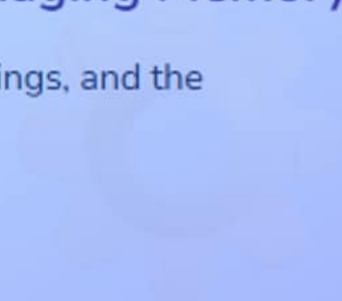
Tool architecture, storage logic, and semantic retrieval



PART 2

Enabling and Managing Memory

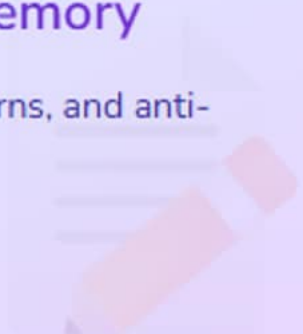
One toggle, VS Code settings, and the management UI



PART 3

What to Store in Memory

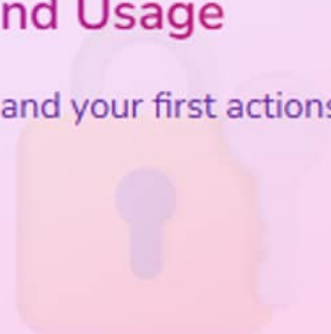
Persistence Test, ideal patterns, and anti-patterns



PART 4

Privacy, Security and Usage

Privacy model, guardrails, and your first actions



Click any section to jump directly there

How Copilot Memory Works

Tool architecture, storage decision logic, and semantic retrieval — the cognitive foundation everything else builds on.



Persistent tool

Memory is a first-class Copilot tool — not a prompt hack or a workaround.



Semantic retrieval

Activates by meaning, not keywords — one preference covers many contexts.



28-day lifecycle

Auto-expires stale memories to prevent silently degraded suggestions.

```
$ copilot memory list  
↳ Viewing stored preferences across sessions
```

What the Agent Stores — and What It Skips

Explicit rule	From now on, always... — direct future-tense instruction, stored every time.	✔ Store
Style pref	I prefer early returns, functional components — lasting personal habit.	✔ Store
Architecture	We use Redis because pub/sub was needed — decision context preserved.	✔ Store
Session task	Fix the bug on line 47 — ephemeral task, irrelevant after this session.	⏮ Skip
Credentials	API key: abc123 — memory syncs across machines; never store secrets.	⛔ Never

Two Operations: Store and Retrieve

Store — Saving What Matters

User: From now on, always ask clarifying questions

Agent evaluates persistence-worthiness of the request

Agent confirms: I will remember this preference

Saved encrypted to your GitHub account across environments

Retrieve — Applying in Context

New session opens — blank chat, no injected context

Memory tool retrieves semantically relevant entries

Matches by meaning, not by exact keywords

Preferences applied automatically from the first response

One Account. Every Environment.



VS Code Chat

Primary workspace — memory stored and recalled across all chat sessions automatically.



GitHub.com

Same memories on the web interface — identical behavior wherever you open a chat.



GitHub CLI

gh copilot inherits your account memories — consistent preferences in the terminal.



Account-linked

Memories follow your GitHub identity, not a machine — switch computers seamlessly.

Freshness Enforced. ROI Compounded.

Stale preferences expire. Important ones survive because you keep using them.

28 days

Auto-expiration window — stale memories purge automatically

GitHub Copilot Docs



Write once

Store a preference in 2 minutes; benefit in every future session indefinitely.



Auto-expires

Stale memories expire to prevent silently degraded suggestions over time.



Zero warm-up

First session after enabling: preferences apply from the first token generated.



Semantic retrieval matches by meaning — not keywords. Write once, benefit in every context.

Enabling and Managing Copilot Memory

Setup is a single JSON toggle — keep this brisk and let the management UI be the main engagement point.



One toggle

A single VS Code setting enables cross-session persistence everywhere.



Management UI

View, edit, and delete stored memories from a dedicated panel.



Cross-platform

Active in VS Code, GitHub.com, and GitHub CLI simultaneously.

```
$ code --open-settings  
↳ Enabling memory in VS Code settings JSON
```

One Setting. Instant Cross-Session Memory.

.vscode/settings.json

```
{
  "github.copilot.chat.copilotMemory.enabled": true
}
```



One toggle

A single JSON setting enables cross-session memory in VS Code immediately.



Multi-platform

VS Code, GitHub.com, and GitHub CLI all activate from the same account setting.



Immediate effect

Active in your next chat session — no restart or additional configuration needed.

Full Control Over What Persists



View

GitHub Settings > Copilot > Memory shows a timestamped list of all stored preferences.



Edit and delete

Remove individual memories when preferences evolve or you switch project context.



Bulk clear

Select multiple or use Clear All to reset — useful after major project transitions.



Export JSON

Download all memories as JSON for backup or audit before making bulk changes.

What to Store in Memory

The decision framework and Persistence Test are the most actionable takeaway — map it to your own workflows.



Ideal patterns

Coding style, toolchain preferences, review standards — anything cross-session.



Anti-patterns

File-specific fixes, temp context, secrets — these degrade quality over time.



Persistence Test

Three questions that determine if something belongs in memory.

```
$ copilot memory add  
↳ Storing a new coding preference interactively
```

Persistence-Worthy Preferences



Coding style

Early returns, TypeScript strict, functional components, JSDoc on public APIs.



Workflow habits

Ask clarifying questions first, show plan before changes, include rollback strategy.



Communication

Bullet points, concise summaries, code examples in explanations, one-sentence preview.



Arch decisions

Redis for pub/sub, Result type for error handling, repository pattern for DB queries.

What Should NOT Be Stored



Session tasks

Fix bugs on specific lines, current file contents, or refactoring tasks in context.



Sensitive data

Passwords, API keys, tokens, connection strings — memory syncs; never store credentials.



Temp shortcuts

Skip tests for now, ignore linting errors — temporary compromises must not persist.



Team standards

Conventions everyone must follow belong in custom instructions, not personal memory.

The Persistence Test



Future sessions?

Will this apply to multiple future conversations?

Yes → store in memory

No → use ephemeral context

Unsure → try and curate later



Personal or team?

Is this your style, or a standard the whole team must follow?

Just you → Memory

Whole team → Custom Instructions

Both → split the layers



Specific enough?

Can the AI apply this consistently and correctly?

Early returns → yes

Write good code → no

Bullet-point explanations → yes

Memory vs. Custom Instructions

Copilot Memory

Personal preferences — follows your GitHub account, not the repo

Private to you — teammates cannot see your memories

Cross-project — same preferences on every codebase

One setting toggle, active in 5 minutes

Custom Instructions

Team standards — shared and version controlled in the repository

Visible to all contributors — enforces team consistency

Repo-scoped — applies within a specific path or project

Markdown file in `.github` — reviewed like any other code

Privacy, Security, and Effective Usage

Front-load the privacy model briefly, then pivot hard into real-world impact and your first actions with memory.



Privacy model

Memories are scoped to you — not shared with teams or used for training.



Real-world impact

Before/after workflow deltas from teams using persistent memory.



First actions

Three things you can do today, this week, and this month.

```
$ copilot memory --help  
↳ Reviewing memory privacy and scope options
```

Encrypted, Private, and Under Your Control

✔ What IS Stored

Text summaries of preferences — not code snippets or file contents

Linked to your GitHub account — no PII beyond account identity

Encrypted at rest and in transit on GitHub infrastructure

28-day auto-expiration — stale preferences purge automatically

🚫 What Is NEVER Stored

File contents, code snippets, or current workspace state

Secrets, API keys, passwords, or connection strings

Personal identifying information beyond your GitHub account

Session-specific paths, line numbers, or ephemeral task context

Enterprise Governance at a Glance



Admin visibility

Memory content is private to each user — admins see usage events only, never memory content.



Sensitive data guard

Tool filters secrets; users can delete any memory; 28-day auto-expiry is a safety net.



Org control

Organizations can disable memory for all users via policy — all-or-nothing governance option.



Data portability

Export all memories as JSON anytime for backup, audit, or deletion before account changes.

The Math Is Simple. The ROI Is Not.

Store once. Benefit every session — indefinitely.

99x

Return on investment — 2 minutes stored, 198 minutes reclaimed across 40 sessions

Example from GitHub Copilot Memory docs

2 min invested

Time to write a clear, specific preference the AI can apply consistently every session.

Forever saves

That preference eliminates re-explanation in every future session — permanently.

Compounds over time

10 stored preferences means 10x the time saved per session vs. explaining from scratch.

Session zero

First chat after enabling: preferences apply from the first token — zero warm-up cost.

 200 minutes wasted re-explaining across 40 sessions. 2 minutes to store it once. That is the compounding power of memory.

Life Before and After Copilot Memory

Before

Re-explain coding style every session

Prompt-engineer preferences into every chat

Context lost when switching between tools

Team members get inconsistent Copilot behavior

After

Style and preferences active from the first token

Zero-prompt context — Copilot already knows

Consistent behavior across VS Code, web, and CLI

Memory plus custom instructions compose cleanly

0 min

Context warm-up per session

2 min

One-time preference investment

∞

Sessions that benefit

✓ What You Can Do Today

Today

- Enable memory in VS Code settings (chat.memory.enabled)
- Open the memory management panel and review defaults
- Write your first preference: preferred language or framework

This Week

- Store your test and review style preferences
- Add framework and dependency preferences
- Tell Copilot your PR and commit message conventions

This Month

- Audit stored memories after 30 days for freshness
- Layer team custom instructions alongside personal memory
- Measure session warm-up time before vs. after adoption

🔑 Key Takeaway

Two minutes of preference-writing eliminates context re-establishment permanently — every session compounds the investment.

Official Documentation

[Copilot Memory documentation](#)

Core concepts, memory curation, and privacy model

[Custom instructions in VS Code](#)

Complementary team-wide instructions feature



Copilot Memory

Your preferences, persisted. Your context, never lost.



Semantic retrieval

Activates by meaning — write once, benefit forever



2-minute investment

Eliminates 5–10 min of context re-setup per session



Memory plus Instructions

Personal style composing cleanly with team standards

Enable memory today. Write your first preference in the next 5 minutes.